

Algoritmi di Base

Algoritmi di Base

Introduzione agli algoritmi di ricerca e ordinamento più comuni, implementati in C++.

Cos'è un algoritmo?

Un **algoritmo** è una sequenza di passi ben definiti per risolvere un problema — come una ricetta di cucina.

Capire gli algoritmi di base ti insegna a pensare in modo strutturato e a valutare quanto "veloce" o "lento" è il tuo codice. Sono fondamentali in informatica.

Vediamo i quattro algoritmi più importanti per chi inizia: ricerca lineare, ricerca binaria, bubble sort e selection sort.

Ricerca Lineare

Come funziona

La **ricerca lineare** è la più semplice: guardi gli elementi uno per uno dall'inizio alla fine, finché non trovi quello che cerchi.

È come cercare il tuo nome in un elenco scritto a mano: leggi il primo nome, poi il secondo, poi il terzo... finché lo trovi o arrivi in fondo.

Implementazione

```
#include <iostream>
using namespace std;

// Cerca target nell'array. Restituisce la posizione (indice) oppure -1 se
// non trovato.
int ricercaLineare(int arr[], int dimensione, int target) {
    for (int i = 0; i < dimensione; i++) {
        if (arr[i] == target) {
            return i;    // trovato! restituisce la posizione
        }
    }
    return -1;          // non trovato
}

int main() {
    int numeri[] = {15, 3, 8, 42, 7, 19, 11};
    int dim = 7;
    int cercato = 42;

    int risultato = ricercaLineare(numeri, dim, cercato);

    if (risultato != -1) {
        cout << "Trovato " << cercato << " alla posizione " << risultato <<
endl;
    } else {
        cout << cercato << " non trovato" << endl;
    }

    return 0;
}
```

Output:

Trovato 42 alla posizione 3

Caratteristiche

- Funziona su array **non ordinati** — l'ordine non importa
- Nel caso peggiore (elemento non presente o all'ultimo posto): controlla tutti gli **n** elementi

- Semplicissimo da scrivere e capire

Ricerca Binaria

Come funziona

La **ricerca binaria** è molto più veloce, ma richiede che l'array sia **già ordinato**.

Pensa a quando cerchi una parola nel dizionario. Non inizi dalla prima pagina — apri il dizionario a metà. Se la parola viene prima, guardi nella prima metà; se viene dopo, guardi nella seconda metà. Ripeti, dimezzando ogni volta lo spazio di ricerca.

Passi:

1. Guarda l'elemento al centro dell'array
2. Se è quello cercato, hai finito
3. Se quello cercato è più piccolo, continua solo nella metà sinistra
4. Se quello cercato è più grande, continua solo nella metà destra
5. Ripeti finché trovi l'elemento o lo spazio si esaurisce

Implementazione

```
#include <iostream>
using namespace std;

int ricercaBinaria(int arr[], int dimensione, int target) {
    int sinistra = 0;
    int destra = dimensione - 1;

    while (sinistra <= destra) {
        int centro = sinistra + (destra - sinistra) / 2;

        if (arr[centro] == target) {
            return centro;          // trovato!
        } else if (arr[centro] < target) {
            sinistra = centro + 1;   // cerca nella metà destra
        } else {
            destra = centro - 1;     // cerca nella metà sinistra
        }
    }

    return -1; // non trovato
}

int main() {
    // IMPORTANTE: l'array deve essere ordinato!
    int numeri[] = {2, 5, 8, 12, 16, 23, 38, 45, 72, 91};
    int dim = 10;
    int cercato = 23;

    int risultato = ricercaBinaria(numeri, dim, cercato);

    if (risultato != -1) {
        cout << "Trovato " << cercato << " alla posizione " << risultato <<
endl;
    } else {
        cout << cercato << " non trovato" << endl;
    }

    return 0;
}
```

Output:

Trovato 23 alla posizione 5

Visualizzazione passo per passo

Cerchiamo 23 in {2, 5, 8, 12, 16, 23, 38, 45, 72, 91} :

Passo 1: centro = 4 $\rightarrow$ arr[4] = 16 < 23 $\rightarrow$ vai a destra (sinistra = 5)

Passo 2: centro = 7 $\rightarrow$ arr[7] = 45 > 23 $\rightarrow$ vai a sinistra (destra = 6)

Passo 3: centro = 5 $\rightarrow$ arr[5] = 23 $\rightarrow$ TROVATO!

Soli 3 passi su un array di 10 elementi. Con un milione di elementi, ne basterebbero circa 20.

Bubble Sort

Come funziona

Il **bubble sort** (ordinamento a bolla) scorre l'array più volte. A ogni passata, confronta gli elementi vicini a coppie e li scambia se sono nell'ordine sbagliato. I valori grandi "salgono" verso la fine, come bolle d'acqua.

Implementazione

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int dimensione) {
    for (int i = 0; i < dimensione - 1; i++) {
        // Ogni passata sposta il valore più grande in fondo
        for (int j = 0; j < dimensione - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                // Scambia i due elementi vicini
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void stampaArray(int arr[], int dimensione) {
    for (int i = 0; i < dimensione; i++) {
        cout << arr[i];
        if (i < dimensione - 1) cout << ", ";
    }
    cout << endl;
}

int main() {
    int numeri[] = {64, 34, 25, 12, 22, 11, 90};
    int dim = 7;

    cout << "Prima dell'ordinamento: ";
    stampaArray(numeri, dim);

    bubbleSort(numeri, dim);

    cout << "Dopo l'ordinamento: ";
    stampaArray(numeri, dim);

    return 0;
}
```

Output:

Prima dell'ordinamento: 64, 34, 25, 12, 22, 11, 90
Dopo l'ordinamento: 11, 12, 22, 25, 34, 64, 90

Visualizzazione di una passata

Array: {64, 34, 25, 12}

64 e 34: $64 > 34 \rightarrow$ scambia $\rightarrow \{34, 64, 25, 12\}$
64 e 25: $64 > 25 \rightarrow$ scambia $\rightarrow \{34, 25, 64, 12\}$
64 e 12: $64 > 12 \rightarrow$ scambia $\rightarrow \{34, 25, 12, 64\}$

Il 64 è finito in fondo. Si ripete per il resto.

Selection Sort

Come funziona

Il **selection sort** (ordinamento per selezione) funziona in modo diverso:

1. Trova il valore più piccolo e mettilo in prima posizione
2. Trova il secondo valore più piccolo e mettilo in seconda posizione
3. Ripeti fino alla fine

Come ordinare le carte: ogni volta "scegli" la carta più bassa e la metti al suo posto.

Implementazione

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int dimensione) {
    for (int i = 0; i < dimensione - 1; i++) {
        // Trova il minimo nella parte non ancora ordinata
        int indiceMinimo = i;
        for (int j = i + 1; j < dimensione; j++) {
            if (arr[j] < arr[indiceMinimo]) {
                indiceMinimo = j;
            }
        }

        // Scambia il minimo trovato con l'elemento in posizione i
        if (indiceMinimo != i) {
            int temp = arr[i];
            arr[i] = arr[indiceMinimo];
            arr[indiceMinimo] = temp;
        }
    }
}

int main() {
    int numeri[] = {64, 25, 12, 22, 11};
    int dim = 5;

    cout << "Prima: ";
    for (int i = 0; i < dim; i++) cout << numeri[i] << " ";
    cout << endl;

    selectionSort(numeri, dim);

    cout << "Dopo: ";
    for (int i = 0; i < dim; i++) cout << numeri[i] << " ";
    cout << endl;

    return 0;
}
```

Output:

Prima: 64 25 12 22 11
Dopo: 11 12 22 25 64

Confronto tra gli algoritmi

Algoritmo	Array ordinato richiesto?	Velocità (caso peggiore)	Difficoltà
Ricerca Lineare	No	N confronti	Facilissima
Ricerca Binaria	Sì	~20 confronti per 1 milione di elementi	Moderata
Bubble Sort	No (ordina lui)	N ² confronti	Facile
Selection Sort	No (ordina lui)	N ² confronti	Facile

- Usa la **ricerca lineare** su array piccoli o non ordinati
- Usa la **ricerca binaria** su array grandi e già ordinati
- **Bubble sort** e **selection sort** sono utili per capire i concetti, ma lenti su array grandi

La soluzione pronta in C++: `std::sort`

Nella pratica, non scrivere mai bubble sort o selection sort in un programma vero. La libreria standard ha `sort()` che è molto più veloce:

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> numeri = {64, 25, 12, 22, 11, 90, 38};

    sort(numeri.begin(), numeri.end()); // ordina in modo efficiente

    cout << "Ordinato: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    // Ricerca binaria già pronta
    if (binary_search(numeri.begin(), numeri.end(), 22)) {
        cout << "22 trovato!" << endl;
    }

    return 0;
}

```

Output:

```

Ordinato: 11 12 22 25 38 64 90
22 trovato!

```

Studiare gli algoritmi "di base" ti insegna a ragionare passo per passo e a capire il concetto di efficienza — due delle basi più importanti dell'informatica.